

Assignment 1

Hadeer Ghoneim

Verification Diploma

Q1

1. Design no edits

```
1  module adder (  
2      input  clk,  
3      input  reset,  
4      input  signed [3:0] A, // Input data A in 2's complement  
5      input  signed [3:0] B, // Input data B in 2's complement  
6      output reg signed [4:0] C // Adder output in 2's complement  
7      );  
8  
9      // Register output C  
10     always @(posedge clk or posedge reset) begin  
11         if (reset)  
12             C <= 5'b0;  
13         else  
14             C <= A + B;  
15     end  
16  
17 endmodule  
18
```

2.verification plan

Label	design requirement description	stimulus generation	functional coverage	functionality check
adder0	reset	Directed	100%	A checker in the testbench to make sure the output is correct
adder1	at A maxpos 3 times and b diffirent between maxpos, maxneg, zero	Directed	100%	A checker in the testbench to make sure the output is correct
adder2	at A maxneg 3 times and b diffirent between maxpos, maxneg, zero	Directed	100%	A checker in the testbench to make sure the output is correct
adder3	at A zero 3 times and b diffirent between maxpos, maxneg, zero	Directed	100%	A checker in the testbench to make sure the output is correct
adder4	extra directed values at [0+1,1+2,1+7,5+4]	Directed	100%	A checker in the testbench to make sure the output is correct

3. Testbench

```
1  module adder_tb ();
2
3      reg signed [3:0] A_tb, B_tb;
4      reg signed [4:0] C_tb;
5      bit clk;
6      bit rst;
7      localparam MAXPOS = 7;
8      localparam MAXNEG = -8;
9      localparam ZERO = 0 ;
10     int error_count;
11     int correct_count;
12
13
14     adder DUT(clk,rst,A_tb,B_tb,C_tb);
15
16     always begin
17         #5 clk = ~clk;
18     end
19
```

```

20  initial begin
21      clk = 0;
22      error_count = 0;
23      correct_count = 0;
24
25      //adder0
26      asser_reset();
27
28      //adder1
29      A_tb = MAXPOS ; B_tb = MAXPOS ; check_result(MAXPOS + MAXPOS);
30      A_tb = MAXPOS ; B_tb = MAXNEG ; check_result(MAXPOS + MAXNEG);
31      A_tb = MAXPOS ; B_tb = ZERO ; check_result(MAXPOS + ZERO);
32
33      //adder2
34      A_tb = MAXNEG ; B_tb = MAXPOS ; check_result(MAXNEG + MAXPOS);
35      A_tb = MAXNEG ; B_tb = MAXNEG ; check_result(MAXNEG + MAXNEG);
36      A_tb = MAXNEG ; B_tb = ZERO ; check_result(MAXNEG + ZERO);
37
38      //adder3
39      A_tb = ZERO ; B_tb = MAXPOS ; check_result(ZERO + MAXPOS);
40      A_tb = ZERO ; B_tb = MAXNEG ; check_result(ZERO + MAXNEG);
41      A_tb = ZERO ; B_tb = ZERO ; check_result(ZERO + ZERO);
42
43      //adder4
44      //extra check to achieve 100% coverage
45      //Add more test vectors
46      A_tb = 0 ; B_tb = 1 ; check_result(1);
47      A_tb = 1 ; B_tb = 2 ; check_result(3);
48      A_tb = 1 ; B_tb = 7 ; check_result(8);
49      A_tb = 5 ; B_tb = 4 ; check_result(9);
50
51      $stop;
52
53  end
54

```

Edit is here : change reset value from 0 to 1 and from 1 to 0

There are 2 other ways to achieve that:

1. Use task asser_reset(); twice times at initial block
2. Exclude reset and do not do that

```

49
50 task asser_reset ();
51     A_tb = 0 ; B_tb = 0;
52     rst =0;
53     check_result(0);
54     @(negedge clk);
55     A_tb = 0 ; B_tb = 0;
56     rst =1;
57     check_result(0);
58     rst =0;
59 endtask
60
61
62 task check_result(input signed [4:0]sum);
63     @(negedge clk);
64
65     if(C_tb != sum)
66         error_count = error_count + 1;
67     else
68         correct_count = correct_count +1;
69 endtask
70
71 endmodule
72

```

4. Do file

```

1  vlib work
2  vlog adder.v adder_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.adder_tb -cover
4  add wave *
5  coverage save adder_tb.ucdb -onexit -du work.adder
6  run -all

```

5. Coverage report text file

```
code_cvr_addr - Notepad
File Edit Format View Help
Coverage Report by instance with details

=====
=== Instance: /\adder_tb#DUT
=== Design Unit: work.adder
=====
Branch Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Branches                  2        2         0    100.00%

=====Branch Details=====

Branch Coverage for instance /\adder_tb#DUT

  Line      Item              Count      Source
  ----      -
  File adder.v
-----IF Branch-----
  11              17    Count coming in to IF
  11              2      if (reset)
  13              15      else

Branch totals: 2 hits of 2 branches = 100.00%

Statement Coverage:
  Enabled Coverage          Bins      Hits      Misses  Coverage
  -----
  Statements                3        3         0    100.00%
```

Statements	3	3	0	100.00%
------------	---	---	---	---------

=====Statement Details=====

Statement Coverage for instance /\adder_tb#DUT --

Line	Item	Count	Source
----	----	-----	-----
File adder.v			
1			module adder (
2			input clk,
3			input reset,
4			input signed [3:0] A, // Input data A in 2's complement
5			input signed [3:0] B, // Input data B in 2's complement
6			output reg signed [4:0] C // Adder output in 2's complement
7			);
8			
9			// Register output C
10	1	17	always @(posedge clk or posedge reset) begin
11			if (reset)
12	1	2	C <= 5'b0;
13			else
14	1	15	C <= A + B;

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

```

8
9                                // Register output C
10           1                   17    always @(posedge clk or posedge reset) begin
11
12           1                   2      C <= 5'b0;
13
14           1                   15    else
15                                   C <= A + B;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	30	30	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\adder_tb#DUT --

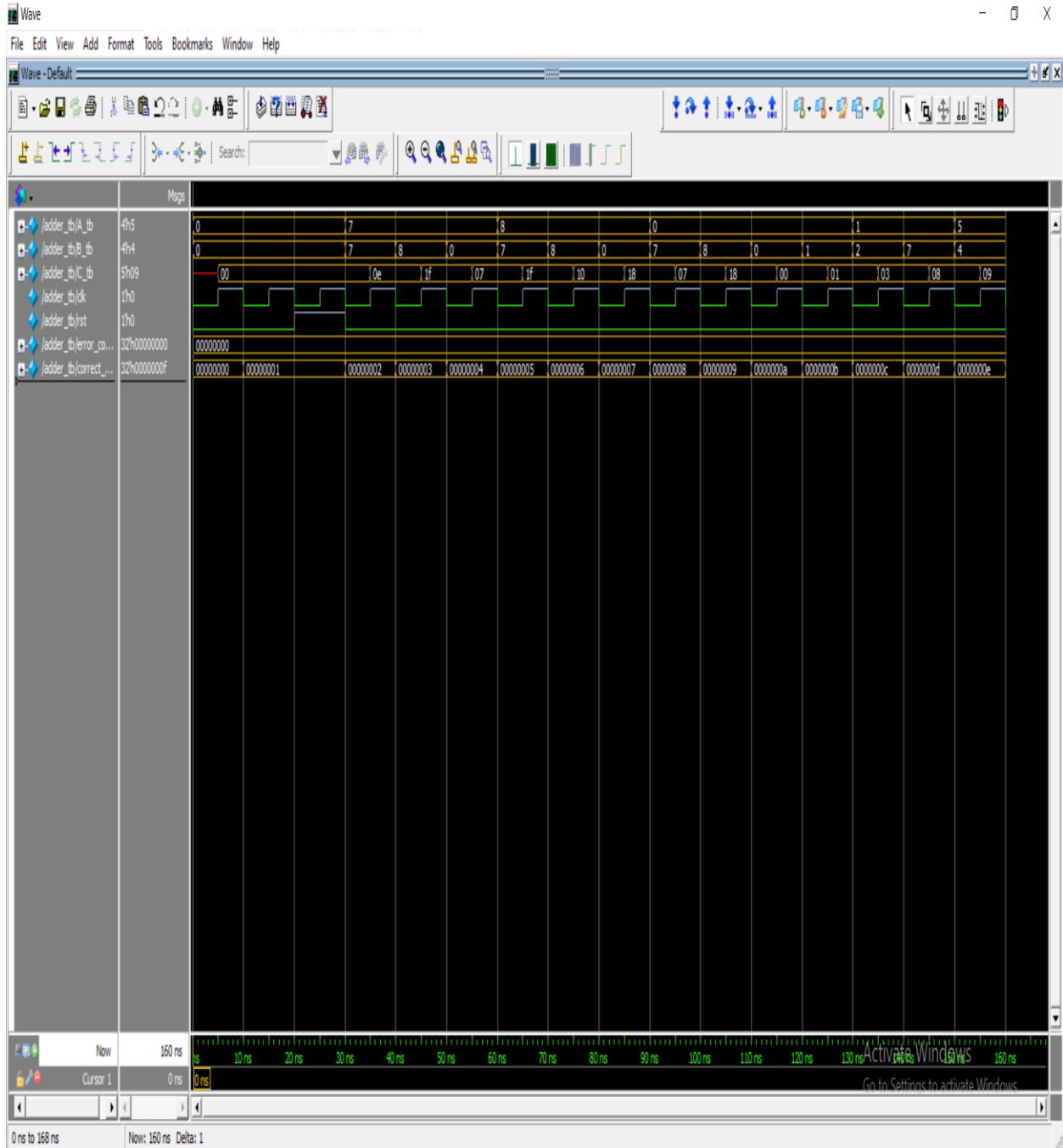
Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
A[0-3]	1	1	100.00
B[0-3]	1	1	100.00
C[4-0]	1	1	100.00
clk	1	1	100.00
reset	1	1	100.00

Total Node Count = 15
Toggled Node Count = 15
Untoggled Node Count = 0

Toggle Coverage = 100.00% (30 of 30 bins)

Total Coverage By Instance (filtered view): 100.00%

1. QuestaSim waveform snippets



Q2

1. DESIGN

Fixed : reg Y “was syntax error” and give 2'b00 and put default case for comparing it with testbench , don't give error count.

```
1  module priority_enc (
2      input  clk,
3      input  rst,
4      input  [3:0] D,
5      output reg [1:0] Y,
6      output reg valid
7  );
8
9  always @(posedge clk) begin
10     if (rst)
11         Y <= 2'b00;
12     else
13         casex (D)
14             4'b1000: Y <= 2'b00;
15             4'bX100: Y <= 2'b01;
16             4'bXX10: Y <= 2'b10;
17             4'bXXX1: Y <= 2'b11;
18             default: Y <= 2'bxx;
19         endcase
20
21     valid <= (~|D)? 1'b0: 1'b1 ;
22 end
23 endmodule
```

2. Verification Plan

Label	design requirement description	stimulus generation	functional coverage	functionality check
COUNTER_1	when the reset is asserted the counter value should be low	directed at the start of the simulation	-	A checker in the testbench to make sure the output is correct
COUNTER_2	when the load is asserted the output count_out should take the value of load_date input	Directed	-	A checker in the testbench to make sure the output is correct
CHECK_1	checks at D = 4'b1000; Y = 0;	Directed	100%	
CHECK_2	checks at 4'bX100: Y <= 1;	Directed	100%	
CHECK_3	checks at 4'bXX10: Y <= 2;	Directed	100%	
CHECK_4	checks at 4'bXXX1: Y <= 3;	Directed	100%	
CHECK_5	checks at default: Y <= 2'bxx;	Directed	100%	
CHECK_6	same as check 2 to transfer from x to 0 and 1 to achieve 100% coverage	directed		

3. Testbench

```

1  module priority_enc_tb ();
2
3      reg [3:0] D_tb;
4      logic [1:0] Y_tb;
5      reg valid_tb;
6      bit clk;
7      bit rst;
8      int error_count;
9      int correct_count;
10
11     priority_enc DUT_enc (clk,rst,D_tb,Y_tb,valid_tb);
12
13     always begin
14         #5 clk = ~clk;
15     end
16
17
18     //COUNTER_1
19     task asser_reset ();
20     @(negedge clk);
21         D_tb = 0 ;
22         rst =1;
23         check_Y(2'b00);
24         check_valid(0);
25         rst =0;
26     endtask
27

```

```

28
29 //COUNTER_2
30 initial begin
31     clk = 0;
32     error_count = 0;
33     correct_count = 0;
34
35     asserr_reset();
36
37     //CHECK_1
38     D_tb = 4'b1000; check_Y(2'b00); check_valid(1);
39     //CHECK_2
40     D_tb = 4'b0100; check_Y(2'b01); check_valid(1);
41     D_tb = 4'b1100; check_Y(2'b01); check_valid(1);
42     //CHECK_3
43     D_tb = 4'b1110; check_Y(2'b10); check_valid(1);
44     D_tb = 4'b1010; check_Y(2'b10); check_valid(1);
45     D_tb = 4'b0110; check_Y(2'b10); check_valid(1);
46     D_tb = 4'b0010; check_Y(2'b10); check_valid(1);
47     //CHECK_4
48     D_tb = 4'b0001; check_Y(2'b11); check_valid(1);
49     D_tb = 4'b1001; check_Y(2'b11); check_valid(1);
50     D_tb = 4'b0101; check_Y(2'b11); check_valid(1);
51     D_tb = 4'b1101; check_Y(2'b11); check_valid(1);
52     D_tb = 4'b0011; check_Y(2'b11); check_valid(1);
53     D_tb = 4'b0111; check_Y(2'b11); check_valid(1);
54     D_tb = 4'b1011; check_Y(2'b11); check_valid(1);
55     D_tb = 4'b1111; check_Y(2'b11); check_valid(1);
56     //CHECK_5
57     D_tb = 4'b0000; check_Y(2'bxx); check_valid(0);
58

```

Even covered all cases for priority encoder .. Added this to that initial block to give 100% coverage same as check 2 to transfer from x to 0 and 1

```

59 //CHECK_6
60 D_tb = 4'b0100; check_Y(2'b01); check_valid(1);
61 D_tb = 4'b1100; check_Y(2'b01); check_valid(1);
62 |
63 $stop;
64
65 end
66

```

```

63
64     task check_Y(input [1:0] Y_out);
65     @(negedge clk);
66
67         if(Y_tb != Y_out )
68             error_count = error_count + 1;
69         else
70             correct_count = correct_count + 1;
71     endtask
72
73
74     task check_valid(input valid_out );
75     @(negedge clk);
76
77         if(valid_tb != valid_out)
78             error_count = error_count + 1;
79         else
80             correct_count = correct_count + 1;
81     endtask
82
83 endmodule

```

4. Do file

```

1  vlib work
2  vlog priority_enc.v priority_enc_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.priority_enc_tb -cover
4  add wave *
5  coverage save priority_enc_tb.ucdb -onexit -du work.priority_enc
6  run -all

```

5. Coverage report text file

code_cvr_enc - Notepad

File Edit Format View Help

Coverage Report by instance with details

```
=====
=== Instance: /\priority_enc_tb#DUT_enc
=== Design Unit: work.priority_enc
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	7	7	0	100.00%

```
=====Branch Details=====
```

Branch Coverage for instance /\priority_enc_tb#DUT_enc

Line	Item	Count	Source
----	----	----	-----
File priority_enc.v			
-----IF Branch-----			
10		27	Count coming in to IF
10	1	2	if (rst)
12	1	25	else

Branch totals: 2 hits of 2 branches = 100.00%

-----CASE Branch-----			
13		25	Count coming in to CASE
14	1	3	4'b1000: Y <= 2'b00;
15	1	6	4'bx100: Y <= 2'b01;
16	1	5	4'bXX10: Y <= 2'b10;
17	1	9	4'bXXX1: Y <= 2'b11;
18	1	2	default: Y <= 2'bxx;

Branch totals: 5 hits of 5 branches = 100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	----	-----
Statements	8	8	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\priority_enc_tb#DUT_enc --

Line	Item	Count	Source
----	----	-----	-----
File priority_enc.v			
1			module priority_enc (
2			input clk,
3			input rst,
4			input [3:0] D,
5			output reg [1:0] Y,
6			output reg valid
7			);
8			
9	1	27	always @(posedge clk) begin
10			if (rst)
11	1	2	Y <= 2'b00;
12			else
13			casex (D)
14	1	3	4'b1000: Y <= 2'b00;
15	1	6	4'bx100: Y <= 2'b01;

```

15          1          6          4'bX100: Y <= 2'b01;
16          1          5          4'bXX10: Y <= 2'b10;
17          1          9          4'bXXX1: Y <= 2'b11;
18          1          2          default: Y <= 2'bxx;
19
20          endcase
21          1          27         valid <= (~|D)? 1'b0: 1'b1 ;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Toggles	18	18	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\priority_enc_tb#DUT_enc --

Node	1H->0L	0L->1H	"Coverage"
D[0-3]	1	1	100.00
Y[1-0]	1	1	100.00
clk	1	1	100.00
rst	1	1	100.00
valid	1	1	100.00

Total Node Count = 9

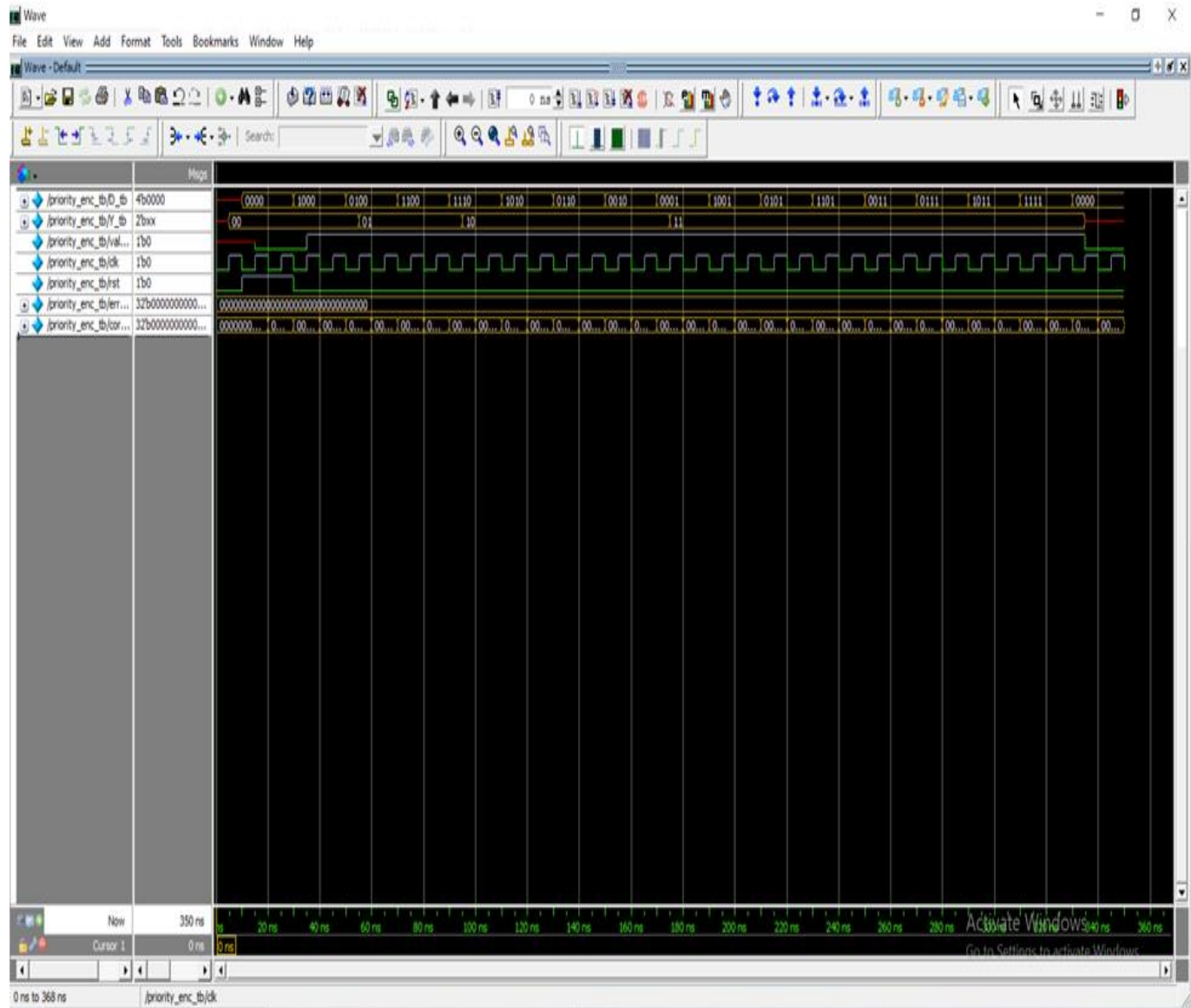
Toggled Node Count = 9

Untoggled Node Count = 0

Toggle Coverage = 100.00% (18 of 18 bins)

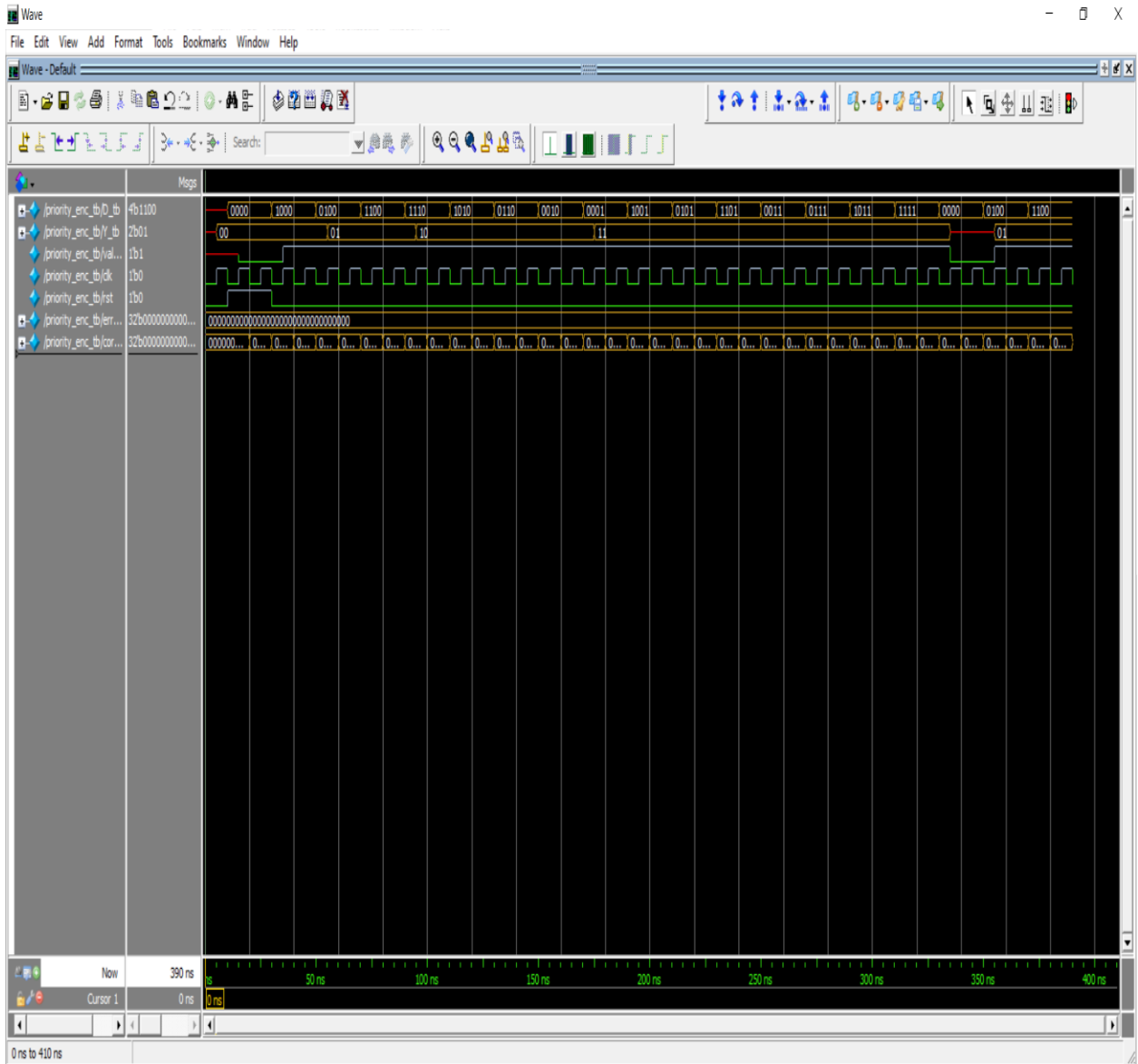
Total Coverage By Instance (filtered view): 100.00%

it achieve specs and truth table



3. QuestaSim waveform snippets

New Waveform cover all priority encoder cases and after adding last check. it has 100% coverage.



Q3

1. Design

```
1  module ALU (  
2      input  clk,  
3      input  reset,  
4      input  [1:0] Opcode,    // The opcode  
5      input  signed [3:0] A,  // Input data A in 2's complement  
6      input  signed [3:0] B,  // Input data B in 2's complement  
7  
8      output reg signed [4:0] C // ALU output in 2's complement  
9  
10     | );  
11  
12     reg signed [4:0]      Alu_out; // ALU output in 2's complement  
13  
14     localparam          Add          = 2'b00; // A + B  
15     localparam          Sub          = 2'b01; // A - B  
16     localparam          Not_A        = 2'b10; // ~A  
17     localparam          ReductionOR_B = 2'b11; // |B  
18
```

```
19     // Do the operation  
20     //covered all cases |  
21     always @* begin  
22         case (Opcode)  
23             Add:          Alu_out = A + B;  
24             Sub:          Alu_out = A - B;  
25             Not_A:        Alu_out = ~A;  
26             ReductionOR_B: Alu_out = |B;  
27             default:      Alu_out = 5'b0;  
28         endcase  
29     end // always @ *  
30
```

```

29
30 // Register output C
31 // why do not remove c ??
32 // why use internal signal alu_out!!!?
33 //maybe to avoid latches???!!
34 //you want alu with register?
35 always @(posedge clk or posedge reset) begin
36     if (reset)
37         C <= 5'b0;
38     else
39         C<= Alu_out;
40 end
41
42 endmodule
43

```

2. Verification Plan

Label	design requirement description	stimulus generation	functional coverage	functionality check
reset	reset	Directed	100%	A checker in the testbench to make sure the output is correct
adder1 & sub1	at A maxpos 3 times and b different between maxpos, maxneg, zero	Directed	100%	A checker in the testbench to make sure the output is correct
adder2 & sub2	at A maxneg 3 times and b different between maxpos, maxneg, zero	Directed	100%	A checker in the testbench to make sure the output is correct
adder3 & sub3	at A zero 3 times and b different between maxpos, maxneg, zero	Directed	100%	A checker in the testbench to make sure the output is correct
adder4 & sub4	extra directed values at [0+1,1+2,1+7,5+4]	Directed	100%	A checker in the testbench to make sure the output is correct
reduction or and not A checker1	checks last 2 opcodes (reduction or and not A)	Directed		
reduction or and not A checker2	checks last 2 opcodes (reduction or and not A)	Directed		
Final sweep	select two values for A and B and apply all opcodes to them.	Directed		

3. Testbench

```
1  module ALU_tb ();
2
3      reg signed [3:0] A_tb, B_tb;
4      logic signed [4:0] C_tb;
5      reg [1:0] Opcode;
6      bit clk;
7      bit rst;
8      localparam MAXPOS = 7;
9      localparam MAXNEG = -8;
10     localparam ZERO = 0 ;
11     int error_count;
12     int correct_count;
13
14     ALU DUT_ALU(clk,rst,Opcode,A_tb,B_tb,C_tb);
15
16     always begin
17         #5 clk = ~clk;
18     end
19
20     initial begin
21         clk = 0;
22         error_count = 0;
23         correct_count = 0;
24
25         //reset
26         asser_reset();
27     end
```

```
29 //adder1 & sub1
30
31 A_tb = MAXPOS ; B_tb = MAXPOS ; check_Add(MAXPOS + MAXPOS);
32 check_Sub(MAXPOS - MAXPOS);
33
34 A_tb = MAXPOS ; B_tb = MAXNEG ; check_Add(MAXPOS + MAXNEG);
35 check_Sub(MAXPOS - MAXNEG);
36
37 A_tb = MAXPOS ; B_tb = ZERO ; check_Add(MAXPOS + ZERO);
38 check_Sub(MAXPOS - ZERO);
39
40
41 //adder2 & sub2
42
43 A_tb = MAXNEG ; B_tb = MAXPOS ; check_Add(MAXNEG + MAXPOS);
44 check_Sub(MAXNEG - MAXPOS);
45
46 A_tb = MAXNEG ; B_tb = MAXNEG ; check_Add(MAXNEG + MAXNEG);
47 check_Sub(MAXNEG - MAXNEG);
48
49 A_tb = MAXNEG ; B_tb = ZERO ; check_Add(MAXNEG + ZERO);
50 check_Sub(MAXNEG - ZERO);
51
52
53 //adder3 & sub3
54
55 A_tb = ZERO ; B_tb = MAXPOS ; check_Add(ZERO + MAXPOS);
56 check_Sub(ZERO - MAXPOS);
57
58 A_tb = ZERO ; B_tb = MAXNEG ; check_Add(ZERO + MAXNEG);
59 check_Sub(ZERO - MAXNEG);
60
61 A_tb = ZERO ; B_tb = ZERO ; check_Add(ZERO + ZERO);
62 check_Sub(ZERO - ZERO);
63
```

```

65 //adder4 & sub4
66 //extra check to achieve 100% coverage
67 //Add more test vectors
68 A_tb = 1 ; B_tb = 0 ; check_Add(1); check_Sub(1);
69 A_tb = 2 ; B_tb = 1 ; check_Add(3); check_Sub(1);
70 A_tb = 7 ; B_tb = 1 ; check_Add(8); check_Sub(6);
71 A_tb = 5 ; B_tb = 4 ; check_Add(9); check_Sub(1);
72
73 // reduction or and not A checker1
74 A_tb = 4; B_tb = 9;
75 check_NotA(~4);
76 check_ReductionOR(|9);
77
78
79 // reduction or and not A checker2
80 A_tb = 6; B_tb = 1;
81 check_NotA(~6);
82 check_ReductionOR(|1);
83
84
85 // Final sweep
86 A_tb = 3; B_tb = 2;
87 check_Add(3 + 2);
88 check_Sub(3 - 2);
89 check_NotA(~3);
90 check_ReductionOR(|2); // reduction OR
91
92 $stop;
93
94 end
95

```

```

96 task asser_reset ();
97
98     A_tb = 0 ; B_tb = 0; Opcode = 0;
99     rst =0;
100     check_Add(0); check_Sub(0);
101     check_NotA(0); check_ReductionOR(0);
102     @(negedge clk);
103     A_tb = 0 ; B_tb = 0; Opcode = 0;
104     rst =1;
105     check_Add(0); check_Sub(0);
106     check_NotA(0); check_ReductionOR(0);
107     rst =0;
108
109 endtask

```

```

113
114 task check_Add(input signed [4:0]sum);
115
116     Opcode = 2'b00; // A + B [Add]
117     @(negedge clk);
118     if(C_tb != sum)
119         error_count = error_count + 1;
120     else
121         correct_count = correct_count +1;
122
123 endtask
124

```

```

126     task check_Sub(input signed [4:0]sub);
127
128         Opcode = 2'b01; // A - B [Sub]
129         @(negedge clk);
130         if(C_tb != sub)
131             error_count = error_count + 1;
132         else
133             correct_count = correct_count +1;
134     endtask
135
136
137
138     task check_NotA(input signed [4:0] inv);
139
140         Opcode = 2'b10; // NOT A
141         @(negedge clk);
142         if (C_tb != inv) error_count++;
143         else correct_count++;
144     endtask
145
146
147
148     task check_ReductionOR(input signed [4:0] red);
149
150         Opcode = 2'b11; // Reduction OR
151         @(negedge clk);
152         if (C_tb != red) error_count++;
153         else correct_count++;
154     endtask
155
156
157
158 endmodule
159

```

4. Do file

```
1  vlib work
2  vlog ALU.v ALU_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.ALU_tb -cover
4  add wave *
5  coverage save ALU_tb.ucdb -onexit -du work.ALU
6  run -all
```

5. Coverage report text file

Excluded Default case

The image displays two screenshots of the 'Code Coverage Analysis' window in a simulation environment. The top screenshot shows 'Statements - by instance (/ALU_tb/DUT_ALU)' for file 'ALU.v'. It lists several code blocks with green checkmarks indicating 100% coverage: an 'always' block at line 21, and conditional blocks for 'Add' (23), 'Sub' (24), 'Not_A' (25), 'ReductionOR_B' (26), and a reset block (36). The 'default' case at line 27 is marked with a red 'E' and a green checkmark, indicating it is excluded from coverage. The bottom screenshot shows 'Branches - by instance (/ALU_tb/DUT_ALU)' for the same file. It shows coverage for the 'Add' (23), 'Sub' (24), 'Not_A' (25), and 'ReductionOR_B' (26) branches. The 'default' branch (27) is again excluded (marked with 'E'). The 'if (reset)' branch (37) and its 'else' branch (39) are both covered with green checkmarks.

Code Coverage Analysis

File Bookmarks Window Help

Code Coverage Analysis

Statements - by instance (/ALU_tb/DUT_ALU)

ALU.v

- ✓ 21 always @* begin
- ✓ 23 Add: Alu_out = A + B;
- ✓ 24 Sub: Alu_out = A - B;
- ✓ 25 Not_A: Alu_out = ~A;
- ✓ 26 ReductionOR_B: Alu_out = |B;
- E 27 default: Alu_out = 5'b0;
- ✓ 36 always @(posedge clk or posedge reset) begin
- ✓ 38 C <= 5'b0;
- ✓ 40 C<= Alu_out;

Code Coverage Analysis

Branches - by instance (/ALU_tb/DUT_ALU)

ALU.v

- ✓ 23 Add: Alu_out = A + B;
- ✓ 24 Sub: Alu_out = A - B;
- ✓ 25 Not_A: Alu_out = ~A;
- ✓ 26 ReductionOR_B: Alu_out = |B;
- E 27 default: Alu_out = 5'b0;
- ✓ 37 if (reset)
- ✓ 39 else

Coverage Report by instance with details

```
=====
=== Instance: /\ALU_tb#DUT_ALU
=== Design Unit: work.ALU
=====
```

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	6	6	0	100.00%

=====Branch Details=====

Branch Coverage for instance /\ALU_tb#DUT_ALU

Line	Item	Count	Source
----	----	-----	-----
File ALU.v			
-----CASE Branch-----			
22		42	Count coming in to CASE
23	1	16	Add: Alu_out = A + B;
24	1	16	Sub: Alu_out = A - B;
25	1	5	Not_A: Alu_out = ~A;
26	1	5	ReductionOR_B: Alu_out = B;

Branch totals: 4 hits of 4 branches = 100.00%

Line	Item	Count	Source
----	----	-----	-----
-----IF Branch-----			
37		43	Count coming in to IF
37	1	4	if (reset)
39	1	39	else

Branch totals: 2 hits of 2 branches = 100.00%

code_cvr_alu_fixed - Notepad

File Edit Format View Help

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	8	8	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\ALU_tb#DUT_ALU --

Line	Item	Count	Source
----	----	-----	-----
File ALU.v			
1			module ALU (
2			input clk,
3			input reset,
4			input [1:0] Opcode, // The opcode
5			input signed [3:0] A, // Input data A in 2's complement
6			input signed [3:0] B, // Input data B in 2's complement
7			
8			output reg signed [4:0] C // ALU output in 2's complement
9			
10			);
11			
12			reg signed [4:0] Alu_out; // ALU output in 2's complement
13			
14			localparam Add = 2'b00; // A + B
15			localparam Sub = 2'b01; // A - B

```

16                                localparam          Not_A              = 2'b10; // ~A
17                                localparam          ReductionOR_B = 2'b11; // |B
18
19                                // Do the operation
20                                //covered all cases no need to use default
21                                1                    42          always @* begin
22                                    case (Opcode)
23                                1                    16          Add:          Alu_out = A + B;
24                                1                    16          Sub:          Alu_out = A - B;
25                                1                    5          Not_A:          Alu_out = ~A;
26                                1                    5          ReductionOR_B: Alu_out = |B;
27                                    default: Alu_out = 5'b0;
28                                endcase
29                                end // always @ *
30
31                                // Register output C
32                                // why do not remove c ??
33                                // why use internal signal alu_out!!!?
34                                //maybe to avoid latches???!!
35                                //you want alu with register?
36                                1                    43          always @(posedge clk or posedge reset) begin

```

```

35                                     //you want alu with register?
36             1                      43         always @(posedge clk or posedge reset) begin
37
38             1                      4           C <= 5'b0;
39
39                                     else
40             1                      39           C<= Alu_out;

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Toggles	44	44	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\ALU_tb#DUT_ALU --

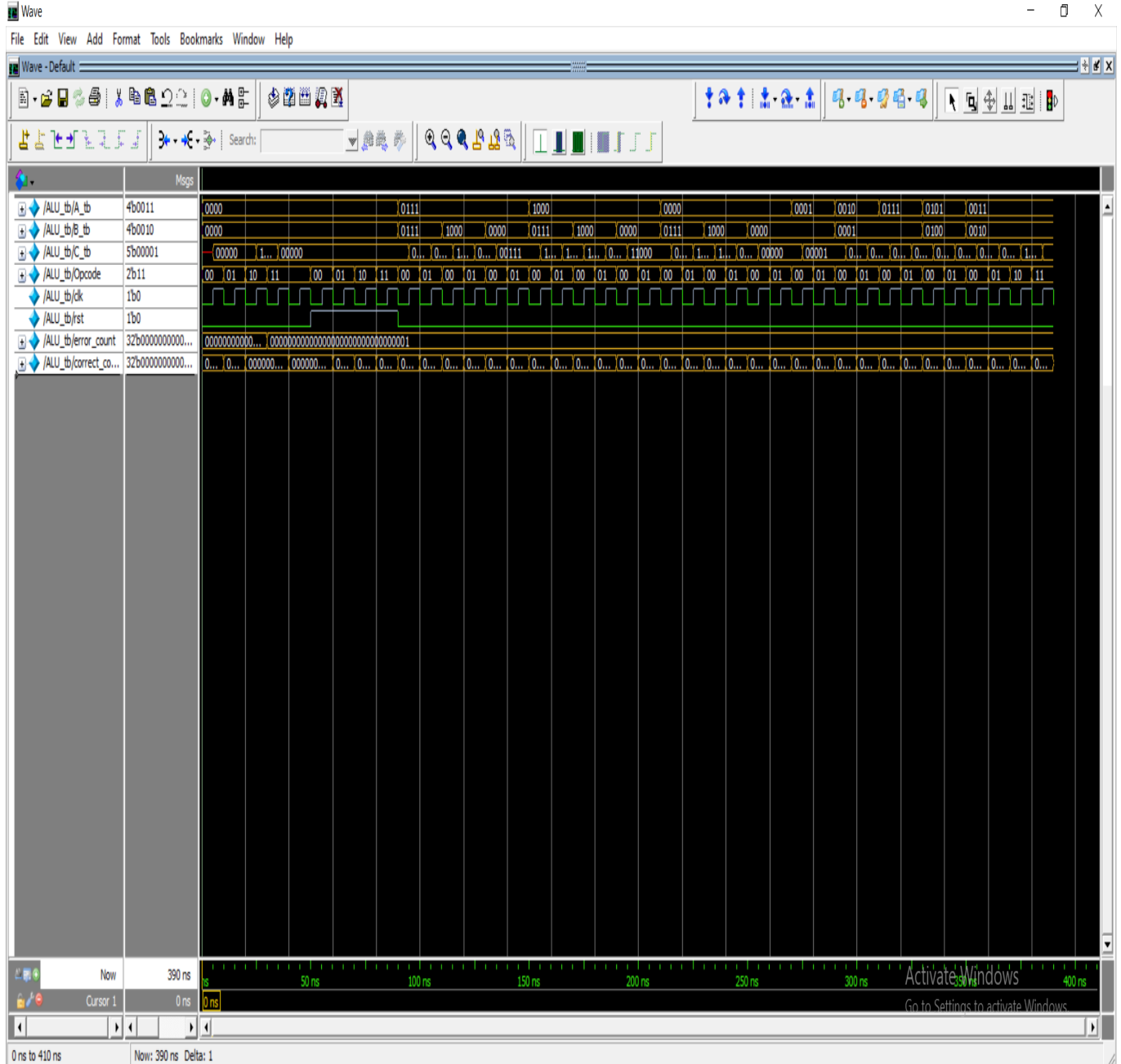
Node	1H->0L	0L->1H	"Coverage"
A[0-3]	1	1	100.00
Alu_out[4-0]	1	1	100.00
B[0-3]	1	1	100.00
C[4-0]	1	1	100.00
Opcode[0-1]	1	1	100.00
clk	1	1	100.00
reset	1	1	100.00

Total Node Count = 22
 Toggled Node Count = 22
 Untoggled Node Count = 0

Toggle Coverage = 100.00% (44 of 44 bins)

Total Coverage By Instance (filtered view): 100.00%

4. Wave form



Q4

1.Design

Edit : change bits number of internal signal: reg mult_out from [47:0] to [35:0]

```
1  module DSP(A, B, C, D, clk, rst_n, P);
2  parameter OPERATION = "ADD";
3  input  [17:0] A, B, D;
4  input  [47:0] C;
5  input clk, rst_n;
6  output reg  [47:0] P;
7
8  reg  [17:0] A_reg_stg1, A_reg_stg2, B_reg, D_reg, adder_out_stg1, adder_out_stg2;
9  reg  [47:0] C_reg;
10 reg  [35:0] mult_out;
11
12 always @(posedge clk or negedge rst_n) begin
13     if (!rst_n) begin // reset
14         A_reg_stg1 <= 0;
15         A_reg_stg2 <= 0;
16         B_reg <= 0;
17         D_reg <= 0;
18         adder_out_stg1 <= 0;
19         mult_out <= 0;
20         P <= 0;
21     end else begin
22         A_reg_stg1 <= A;
23         A_reg_stg2 <= A_reg_stg1;
24         B_reg <= B;
25         C_reg <= C;
26         D_reg <= D;
27         adder_out_stg2 <= adder_out_stg1;
28         if (OPERATION == "ADD") begin
29             adder_out_stg1 <= D_reg + B_reg;
30             P <= mult_out + C_reg;
31         end else if (OPERATION == "SUBTRACT") begin
32             adder_out_stg1 <= D_reg - B_reg;
33             P <= mult_out - C_reg; end
34         mult_out <= A_reg_stg2 * adder_out_stg2;
35     end
36 end
37 endmodule
```

Golden model design

```
1  module dsp_block
2  #( parameter OPERATION = "ADD") // or "SUBTRACT"
3  (   input wire clk,
4      input wire rst_n,
5      input wire [17:0] A,
6      input wire [17:0] B,
7      input wire [17:0] D,
8      input wire [47:0] C,
9      output reg [47:0] P );
10
11     // Internal pipeline registers
12     reg [35:0] multiplier_out;
13     reg [17:0] D_minus_B;
14     reg [47:0] sub_result;
15
16     always @(posedge clk) begin
17
18         if (!rst_n) begin
19             multiplier_out <= 36'b0;
20             P <= 48'b0;
21         end
22
23         else begin
24             multiplier_out <= A * B;
25
26             if (OPERATION == "ADD") begin
27                 P <= multiplier_out + C;
28             end
29             else begin // SUBTRACT
30                 D_minus_B <= D - B;
31                 sub_result <= D_minus_B - C[17:0];
32                 // You may need to extend properly
33                 P <= sub_result;
34             end
35         end
36     end
37 endmodule
```

2.verification plan

Label	design requirement description	stimulus generation	functional coverage	functionality check
COUNTER_1	when the reset is asserted the counter value should be low	directed at the start of the simulation	100%	A checker in the testbench to make sure the output is correct
COUNTER_2	when the load is asserted the output count_out should take the value of load_date input	Randomization	100%	A checker in the testbench to make sure the output is correct

3.testbench

```
1  module DSP_tb();
2
3      reg clk = 0;
4      reg rst_n;
5      reg [17:0] A, B, D;
6      reg [47:0] C;
7      wire [47:0] P_GM, P_TB;
8
9      localparam MAXPOS = 7;
10     localparam MAXNEG = -8;
11     localparam ZERO = 0 ;
12     int error_count;
13     int correct_count;
14
15     DSP #(.OPERATION("ADD")) DUT_DSP (
16         .clk(clk),
17         .rst_n(rst_n),
18         .A(A), .B(B), .D(D), .C(C),
19         .P(P_TB)
20     );
21
22     dsp_block #(.OPERATION("ADD")) GM_DSP (
23         .clk(clk),
24         .rst_n(rst_n),
25         .A(A), .B(B), .D(D), .C(C),
26         .P(P_GM)
27     );
28
29     always begin
30         #5 clk = ~clk;
31     end
32
```

```
33     integer i;
34     initial begin
35
36         clk = 0;
37         error_count = 0;
38         correct_count = 0;
39
40         //COUNTER_1
41         asser_reset();
42
43         //COUNTER_2
44         for(i=0; i<99; i=i+1)begin
45             A = $random;
46             B = $random;
47             D = $random;
48             C = $random;
49             check_result(P_GM);
50         end
51
52         $stop();
53
54     end
55
```

```

54
55  task asser_reset ();
56      A=0; B=0; C=0; D=0;
57      rst_n =1;
58      check_result(0);
59  @(negedge clk);
60      A=0; B=0; C=0; D=0;
61      rst_n =0;
62      check_result(0);
63      rst_n =1;
64  endtask
65
66
67  task check_result(input signed [47:0]P_GM);
68      #4
69      if(P_TB != P_GM)
70          error_count = error_count + 1;
71      else
72          correct_count = correct_count +1;
73  endtask
74
75  endmodule

```

4.do file

```

1  vlib work
2  vlog DSP.v DSP_Block.v DSP_tb.svh +cover -covercells
3  vsim -voptargs=+acc work.DSP_tb -cover
4  add wave *
5  coverage save DSP_tb.ucdb -onexit -du work.DSP
6  run -all

```

5.coverage report

code_cvr_dsp - Notepad

File Edit Format View Help

Coverage Report by instance with details

=====

=== Instance: /\DSP_tb#DUT_DSP

=== Design Unit: work.DSP

=====

Branch Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Branches	2	2	0	100.00%

=====Branch Details=====

Branch Coverage for instance /\DSP_tb#DUT_DSP

NOTE: The modification timestamp for source file 'DSP.v' has been altered since compilation.

Line	Item	Count	Source
----	----	----	-----
File DSP.v			
-----IF Branch-----			
13		42	Count coming in to IF
13	1	1	if (!rst_n) begin // reset
23	1	41	A_reg_stg2 <= A_reg_stg1;

Branch totals: 2 hits of 2 branches = 100.00%

Statement Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Statements	17	17	0	100.00%

=====Statement Details=====

Statement Coverage for instance /\DSP_tb#DUT_DSP --

NOTE: The modification timestamp for source file 'DSP.v' has been altered since compilation.

Line	Item	Count	Source
----	----	----	-----
File DSP.v			
1			module DSP(A, B, C, D, clk, rst_n, P);

```
2           parameter OPERATION = "ADD";
3           input  [17:0] A, B, D;
4           input  [47:0] C;
5           input  clk, rst_n;
6           output reg [47:0] P;
7
8           reg [17:0] A_reg_stg1, A_reg_stg2, B_reg, D_reg, adder_out_stg1, adder_out_stg2;
9           reg [47:0] C_reg;
10          reg [35:0] mult_out;
11
12          1          42  always @(posedge clk or negedge rst_n) begin
13                      if (!rst_n) begin // reset
14                          A_reg_stg1 <= 0;
15                          1          1          A_reg_stg2 <= 0;
16                          1          1          B_reg <= 0;
17                          1          1          D_reg <= 0;
18                          1          1          adder_out_stg1 <= 0;
19                          1          1          mult_out <= 0;
20                          1          1          P <= 0;
21                          1          1          end else begin
22                              A_reg_stg1 <= A;
```

```

23          A_reg_stg2 <= A_reg_stg1;
24          1          41          B_reg <= B;
25          1          41          C_reg <= C;
26          1          41          D_reg <= D;
27          1          41          adder_out_stg2 <= adder_out_stg1;
28          1          41          if (OPERATION == "ADD") begin
29          1          41              adder_out_stg1 <= D_reg + B_reg;
30              P <= mult_out + C_reg;
31          1          41          end else if (OPERATION == "SUBTRACT") begin
32          1          41              adder_out_stg1 <= D_reg - B_reg;
33              P <= mult_out - C_reg; end
34              mult_out <= A_reg_stg2 * adder_out_stg2;
35              end
36              end
37              endmodule

```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
Toggles	688	688	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\DSP_tb#DUT_DSP --

Node	1H->0L	0L->1H	"Coverage"
A[0-17]	1	1	100.00

code_cvr_dsp - Notepad

File Edit Format View Help

```
35                                     end
36                                     end
37                                     endmodule
```

Toggle Coverage:

Enabled Coverage	Bins	Hits	Misses	Coverage
-----	----	----	-----	-----
Toggles	688	688	0	100.00%

=====Toggle Details=====

Toggle Coverage for instance /\DSP_tb#DUT_DSP --

Node	1H->0L	0L->1H	"Coverage"
-----	-----	-----	-----
A[0-17]	1	1	100.00
A_reg_stg1[17-0]	1	1	100.00
A_reg_stg2[17-0]	1	1	100.00
B[0-17]	1	1	100.00
B_reg[17-0]	1	1	100.00
C[0-47]	1	1	100.00
C_reg[47-0]	1	1	100.00
D[0-17]	1	1	100.00
D_reg[17-0]	1	1	100.00
P[47-0]	1	1	100.00
adder_out_stg1[17-0]	1	1	100.00
adder_out_stg2[17-0]	1	1	100.00
clk	1	1	100.00
mult_out[35-0]	1	1	100.00
rst_n	1	1	100.00

Total Node Count = 344
Toggled Node Count = 344
Untoggled Node Count = 0

Toggle Coverage = 100.00% (688 of 688 bins)

Total Coverage By Instance (filtered view): 100.00%

6.Waveform

